

创建杂项设备代码

```
#include <linux/module.h>

#include <linux/fs.h>

#include <linux/uaccess.h>

#include <linux/miscdevice.h>

// misc 设备打开时调用的函数

static int misc_open(struct inode *inode, struct file *file)

{

    printk(KERN_INFO "misc 设备已打开\n"); // 打印内核信息日志，提示设备已打开

    return 0; // 返回 0 表示成功

}

// misc 设备关闭时调用的函数

static int misc_release(struct inode *inode, struct file *file)

{

    printk(KERN_INFO "misc 设备已释放\n"); // 打印内核信息日志，提示设备已释放

    return 0; // 返回 0 表示成功

}

// 从 misc 设备读取数据时调用的函数

static ssize_t misc_read(struct file *file, char __user *buf, size_t count, loff_t *ppos)

{

    printk(KERN_INFO "misc 设备读取操作\n"); // 打印内核信息日志，提示正在进行读取操作

    return 0; // 返回 0 表示没有数据可读
```

```
}
```

```
// 向 misc 设备写入数据时调用的函数
```

```
static ssize_t misc_write(struct file *file, const char __user *buf,  
size_t count, loff_t *ppos)
```

```
{
```

```
    printk(KERN_INFO "misc 设备写入操作\n"); // 打印内核信息日志, 提示  
    正在进行写入操作
```

```
    return count; // 返回写入的数据量
```

```
}
```

```
// 文件操作结构体, 包含 misc 设备的所有操作函数
```

```
static const struct file_operations fops = {
```

```
    .owner = THIS_MODULE, // 模块所有者, 指向当前模块
```

```
    .open = misc_open,    // 设备打开操作
```

```
    .release = misc_release, // 设备释放操作
```

```
    .read = misc_read,    // 设备读取操作
```

```
    .write = misc_write, // 设备写入操作
```

```
};
```

```
// misc 设备结构体, 定义设备的基本信息
```

```
static struct miscdevice misc_dev = {
```

```
    .minor = MISC_DYNAMIC_MINOR, // 使用动态分配的次设备号
```

```
    .name = "jy", // 设备名称
```

```
    .fops = &fops, // 文件操作结构体指针
```

```
};
```

```

// 模块初始化函数，在模块加载时调用
static int __init misc_init(void)
{
    misc_register(&misc_dev); // 注册 misc 设备
    return 0; // 返回 0 表示成功
}

// 模块退出函数，在模块卸载时调用
static void __exit misc_exit(void)
{
    misc_deregister(&misc_dev); // 注销 misc 设备
}

// 指定模块的初始化和退出函数
module_init(misc_init);
module_exit(misc_exit);

// 模块元数据
MODULE_LICENSE("GPL");
MODULE_AUTHOR("JY");

```

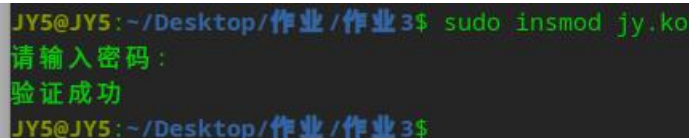


```

JY5@JY5:~/Desktop/作业/作业3$ make
make -C /usr/src/linux-headers-5.10.0-amd64-desktop M=/home/JY5/Desktop/作业/作业3 CC=gcc OBJDUMP=objdump LD=ld modules
make[1]: 进入目录 "/usr/src/linux-headers-5.10.0-amd64-desktop"
arch/x86/Makefile:147: CONFIG_X86_X32 enabled but no binutils support
CC [M] /home/JY5/Desktop/作业/作业3/jy.o
MODPOST /home/JY5/Desktop/作业/作业3/Module.symvers
CC [M] /home/JY5/Desktop/作业/作业3/jy.mod.o
LD [M] /home/JY5/Desktop/作业/作业3/jy.ko
make[1]: 离开目录 "/usr/src/linux-headers-5.10.0-amd64-desktop"

```

图 一 使用 make 运行代码生成内核模块



```

JY5@JY5:~/Desktop/作业/作业3$ sudo insmod jy.ko
请输入密码:
验证成功
JY5@JY5:~/Desktop/作业/作业3$

```

图 二 将内核模块加载到内核中

```
JY5@JY5:~/Desktop/作业/作业3$ lsmod
Module                Size  Used by
jy                     16384  0
```

图 三 查看当前已加载的内核模块

```
JY5@JY5:~/Desktop/作业/作业3$ ls -l /dev/*jy*
crw-rw-rw- 1 root root 10, 51 10月 30 18:47 /dev/jy
```

图 四 查看设备文件

设备测试代码

```
#include <stdio.h>

#include <fcntl.h>

#include <unistd.h>

#include <errno.h>

#include <string.h>

#include <sys/stat.h>

int main() {

    int fd;

    char buf[] = "Hello World";

    ssize_t bytes_written, bytes_read;

    // 检查设备文件是否存在

    if (access("/dev/jy", F_OK) == -1) {

        perror("设备文件不存在");

        return -1;

    } else {

        printf("设备文件存在。\\n");

    }

    // 打开设备文件

    fd = open("/dev/jy", O_RDWR);

    if (fd == -1) {

        perror("打开设备失败");

        return -1;

    }

}
```

```

    } else {
        printf("设备打开成功。\\n");
    }

    // 写入数据到设备

    bytes_written = write(fd, buf, sizeof(buf) - 1); // -1 以排除字符
串结束符

    if (bytes_written == -1) {
        perror("写入设备失败");
        close(fd);
        return -1;
    } else {
        printf("写入设备成功, 写入字节数: %ld.\\n", bytes_written);
    }

    // 从设备读取数据

    lseek(fd, 0, SEEK_SET); // 将文件偏移量重置到文件开头

    bytes_read = read(fd, buf, sizeof(buf) - 1);

    if (bytes_read == -1) {
        perror("从设备读取数据失败");
        close(fd);
        return -1;
    } else {
        printf("从设备读取数据成功, 读取字节数: %ld.\\n", bytes_read);
    }

    // 打印读取的数据

    printf("从设备读取的数据: %s\\n", buf);

```

```

// 关闭设备文件

close(fd);

printf("设备关闭成功。\\n");

return 0;

}

```

```

root@JY5:/home/JY5/Desktop/作业/作业3/test# ./test
设备文件存在。
设备打开成功。
写入设备成功，写入字节数：11。
从设备读取数据成功，读取字节数：0。
从设备读取的数据：Hello World
设备关闭成功。

```

图 五 运行结果

```

2024-10-30 19:06:25 JY5 kernel: [ 3504.361586] misc设备已打开
2024-10-30 19:06:25 JY5 kernel: [ 3504.361620] misc设备写入操作
2024-10-30 19:06:25 JY5 kernel: [ 3504.361655] misc设备读取操作
2024-10-30 19:06:25 JY5 kernel: [ 3504.361662] misc设备已释放

```

图 六 查看 Linux 内核日志，看到了创建的杂项设备的使用